```
import os
import sys
import time
import pytest
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.support.ui import WebDriverWait
LOGIN_URL =
"https://accounts.google.com/signin/v2/identifier?service=mail&flowName=Gl
obalSignIn"
TIMEOUT = 15
MAX_LOAD_TIME_SEC = 5.0
MAX_STEP_TIME_SEC = 3.0
VALID_USER = os.getenv("sharma9103867592@gmail.com")
VALID_PASS = os.getenv("**** **** **** ****")
INVALID_USER = "no.such.user.zz9q8w7e6r5t4y3u2i1o@gmail.com"
WRONG_PASS = "Wrong-Password-123!"
IDENTIFIER_INPUT = (By.ID, "identifierId")
IDENTIFIER_NEXT = (By.ID, "identifierNext")
PASSWORD_INPUT = (By.CSS_SELECTOR, "input[type='password']")
PASSWORD_NEXT = (By.ID, "passwordNext")
ERROR_MSG = (By.CSS_SELECTOR, "div[aria-live='assertive'], div.o6cuMc,
div.Ekjuhf")
FORGOT_EMAIL = (By.XPATH, "//button[contains(.,'Forgot email')]")
FORGOT_PASSWORD = (By.XPATH, "//button[contains(.,'Forgot password')]")
class ResultCounter:
    def __init__(self):
        self.passed = 0
        self.failed = 0
        self.skipped = 0
    def pytest_runtest_logreport(self, report):
        if report.when == "call":
            if report.passed:
                self.passed += 1
            elif report.failed:
                self.failed += 1
        elif report.when == "setup":
            if report.skipped:
                self.skipped += 1
            elif report.failed:
                self.failed += 1
    def pytest_sessionfinish(self, session, exitstatus):
        total = self.passed + self.failed + self.skipped
        print(f"\n{self.passed} of {total} test scenarios passed
({self.failed} failed, {self.skipped} skipped)")
@pytest.fixture
def driver():
    options = webdriver.ChromeOptions()
    options.add_argument("--start-maximized")
    options.add_argument("--lang=en-US")
    drv = webdriver.Chrome(options=options)
    drv.implicitly_wait(0)
    yield drv
```

```
    drv.quit()
def wait_visible(driver, locator, timeout=TIMEOUT):
    return WebDriverWait(driver,
timeout).until(EC.visibility_of_element_located(locator))
def wait_clickable(driver, locator, timeout=TIMEOUT):
    return WebDriverWait(driver,
timeout).until(EC.element_to_be_clickable(locator))
def open_login(driver):
    driver.get(LOGIN_URL)
    return wait_visible(driver, IDENTIFIER_INPUT)
def submit_identifier(driver, identifier):
    field = wait_visible(driver, IDENTIFIER_INPUT)
    field.clear()
    if identifier:
        field.send_keys(identifier)
    wait_clickable(driver, IDENTIFIER_NEXT).click()
def submit_password(driver, password):
    field = wait_visible(driver, PASSWORD_INPUT)
    field.clear()
    if password:
        field.send_keys(password)
    wait_clickable(driver, PASSWORD_NEXT).click()
def get_error_text(driver, timeout=TIMEOUT):
    def _has_text(d):
        for el in d.find_elements(*ERROR_MSG):
            if el.text.strip():
                return el.text.strip()
        return False
    return WebDriverWait(driver, timeout).until(_has_text)
def test_ts01_signin_page_opens(driver):
    field = open_login(driver)
    assert "accounts.google.com" in driver.current_url
    assert "sign in" in driver.title.lower() or "gmail" in
driver.title.lower()
    assert field.is_displayed()
    assert wait_visible(driver, IDENTIFIER_NEXT).is_displayed()
@pytest.mark.skipif(not (VALID_USER and VALID_PASS),
                    reason="Set GMAIL_USER and GMAIL_PASS env vars to
run")
def test_ts02_login_valid_credentials(driver):
    open_login(driver)
    submit_identifier(driver, VALID_USER)
    submit_password(driver, VALID_PASS)
    WebDriverWait(driver, 30).until(
        lambda d: "mail.google.com" in d.current_url or
"myaccount.google.com" in d.current_url
    )
    assert "accounts.google.com/signin" not in driver.current_url
def test_ts03_empty_identifier(driver):
    open_login(driver)
    submit_identifier(driver, "")
    msg = get_error_text(driver)
    assert "enter an email or phone number" in msg.lower()
    assert wait_visible(driver, IDENTIFIER_INPUT).is_displayed()
```

```
def test_ts04_invalid_identifier(driver):
    open_login(driver)
    submit_identifier(driver, INVALID_USER)
    msg = get_error_text(driver)
    assert "couldn't find your google account" in msg.lower() or
"couldnâ□□t find your google account" in msg.lower()
    assert not driver.find_elements(*PASSWORD_INPUT)
@pytest.mark.skipif(not VALID_USER, reason="Set GMAIL_USER env var to
run")
def test_ts05_incorrect_password(driver):
    open_login(driver)
    submit_identifier(driver, VALID_USER)
    submit_password(driver, WRONG_PASS)
    msg = get_error_text(driver)
    assert "wrong password" in msg.lower()
    assert "mail.google.com" not in driver.current_url
@pytest.mark.skipif(not VALID_USER, reason="Set GMAIL_USER env var to
run")
def test_ts06_empty_password(driver):
    open_login(driver)
    submit_identifier(driver, VALID_USER)
    submit_password(driver, "")
    msg = get_error_text(driver)
    assert "enter a password" in msg.lower()
    assert wait_visible(driver, PASSWORD_INPUT).is_displayed()
@pytest.mark.skipif(not VALID_USER, reason="Set GMAIL_USER env var to
run")
def test_ts07_navigation_to_password_step(driver):
    open_login(driver)
    submit_identifier(driver, VALID_USER)
    pwd = wait_visible(driver, PASSWORD_INPUT)
    assert pwd.is_displayed()
    assert pwd.get_attribute("type") == "password"
    assert VALID_USER.lower() in driver.page_source.lower()
    assert wait_visible(driver, PASSWORD_NEXT).is_displayed()
def test_ts08a_forgot_email_flow(driver):
    open_login(driver)
    wait_clickable(driver, FORGOT_EMAIL).click()
    WebDriverWait(driver,
TIMEOUT).until(EC.url_contains("signin/usernamerecovery"))
    assert "recovery" in driver.current_url.lower()
@pytest.mark.skipif(not VALID_USER, reason="Set GMAIL_USER env var to
run")
def test_ts08b_forgot_password_flow(driver):
    open_login(driver)
    submit_identifier(driver, VALID_USER)
    wait_clickable(driver, FORGOT_PASSWORD).click()
    WebDriverWait(driver, TIMEOUT).until(
        lambda d: "recovery" in d.current_url.lower() or "challenge" in
d.current_url.lower()
    )
    assert "accounts.google.com" in driver.current_url
def test_ts09_network_interruption(driver):
    open_login(driver)
```

```
    field = wait_visible(driver, IDENTIFIER_INPUT)
    field.send_keys(INVALID_USER)
    driver.set_network_conditions(offline=True, latency=0, throughput=0)
    try:
        wait_clickable(driver, IDENTIFIER_NEXT).click()
        time.sleep(3)
        assert not driver.find_elements(*PASSWORD_INPUT)
        assert driver.find_element(*IDENTIFIER_INPUT).is_displayed()
    finally:
        driver.set_network_conditions(offline=False, latency=5,
throughput=500 * 1024)
    driver.get(LOGIN_URL)
    assert wait_visible(driver, IDENTIFIER_INPUT).is_displayed()
def test_ts10a_page_load_time(driver):
    start = time.perf_counter()
    driver.get(LOGIN_URL)
    wait_visible(driver, IDENTIFIER_INPUT)
    elapsed = time.perf_counter() - start
    print(f"\nLogin page load: {elapsed:.2f}s (limit
{MAX_LOAD_TIME_SEC}s)")
    assert elapsed <= MAX_LOAD_TIME_SEC
@pytest.mark.skipif(not VALID_USER, reason="Set GMAIL_USER env var to
run")
def test_ts10b_identifier_to_password_step_time(driver):
    open_login(driver)
    field = wait_visible(driver, IDENTIFIER_INPUT)
    field.send_keys(VALID_USER)
    btn = wait_clickable(driver, IDENTIFIER_NEXT)
    start = time.perf_counter()
    btn.click()
    wait_visible(driver, PASSWORD_INPUT)
    elapsed = time.perf_counter() - start
    print(f"\nIdentifier -> password step: {elapsed:.2f}s (limit
{MAX_STEP_TIME_SEC}s)")
    assert elapsed <= MAX_STEP_TIME_SEC
if __name__ == "__main__":
    sys.exit(pytest.main([__file__, "-v", "-s"],
plugins=[ResultCounter()]))
```